

Exercise Sheet 3, 2026

6.0 VU Advanced Database Systems

Felix Kapfer (12429669)

June 16, 2026

Exercise 1 (Scalar and Vector Clocks) [2 Points]

There are three processes provided which are: a , b , and c that exchange five messages $m_1, \dots, m_5$ within a small observation window. The processes have already been running before the window starts, which is why the initial counter values are not zero. The task is to determine the version stamps at the marked points in time, once using *scalar* (Lamport) clocks in part (a) and once using *vector* clocks in part (b). In both cases we additionally report all clock values before and after every send and receive event, so that the four highlighted boxes follow directly from the complete timeline.

Since the two diagrams use *different* message routings and *different* initial values, we treat them as two independent scenarios. The message routing that we read from the two diagrams is summarised in Table 1.

Message	m_1	m_2	m_3	m_4	m_5
Part (a) – scalar	$b \rightarrow a$	$b \rightarrow a$	$a \rightarrow c$	$a \rightarrow b$	$c \rightarrow b$
Part (b) – vector	$b \rightarrow c$	$a \rightarrow c$	$c \rightarrow a$	$b \rightarrow a$	$c \rightarrow b$

Table 1: Sender and receiver of each message, as read from the two diagrams.

Solution 1a) – Scalar (Lamport) clocks

We use the standard Lamport clock rules. Each process keeps a single integer counter, and the counter is updated according to the following two rules:

- Before a process executes a *send* (or an internal event), it increments its own counter by one and attaches the new value to the message.
- On *receiving* a message carrying timestamp t , a process sets its counter to $\max(\text{local}, t) + 1$.

In other words this can be illustrated with the following formula:

$$\text{clock} := \max(\text{local_clock}, t) + 1$$

The initial counters are provided as following:

- $a = 1$
- $b = 4$
- $c = 3$

So next, we use the message directions from the diagram. The five messages are exchanged as the following:

Message	Sender	Receiver
m1	process b	process a
m2	process b	process a
m3	process a	process c
m4	process a	process b
m5	process c	process b

Table 2: Message directions

In order to obtain the values at the marked points, we process the events in their order, which means that we always evaluate a send before the matching receive. The resulting per-process timeline is shown below; the value after each event is the counter that the process holds at that moment.

Process b (start = 4):

Process b starts with counter value 4. First, process b sends message m_1 . Before sending, it increments its counter:

$$b : 4 \rightarrow 5$$

Therefore, message m_1 carries timestamp 5.

Next, process b sends message m_2 . Again, the counter is incremented before sending:

$$b : 5 \rightarrow 6$$

Therefore, message m_2 carries timestamp 6. At this point, the middle marked point on process b is reached. Hence, the value in the middle circle on process b is:

$$b_{\text{mid}} = 6$$

Afterwards, process b receives message m_4 . Message m_4 carries timestamp 9, as derived below. Since the local counter of b is currently 6, it is updated as follows:

$$b := \max(6, 9) + 1 = 10$$

Finally, process b receives message m_5 . Message m_5 also carries timestamp 9. The local counter of b is now already 10, because m_4 was received before m_5 . Therefore, the update is:

$$b := \max(10, 9) + 1 = 11$$

Thus, the value in the right circle on process b is:

$$b_{\text{right}} = 11$$

To summarize the values are as the following:

- send m_1 : $4 \rightarrow 5$, hence m_1 carries timestamp 5.
- send m_2 : $5 \rightarrow 6$, hence m_2 carries timestamp 6.
- point in time: counter = 6 (middle box on b , after sending m_2).
- receive m_4 (timestamp 9): $\max(6, 9) + 1 = 10$.
- receive m_5 (timestamp 9): $\max(10, 9) + 1 = 11$.
- point in time: counter = 11 (right box on b , after receiving m_5).

Process a (start = 1):

Process a starts with counter value 1. First, process a receives message m_1 , which carries timestamp 5. Therefore, process a updates its local counter as follows:

$$a := \max(1, 5) + 1 = 6$$

Next, process a sends message m_3 . Before sending, process a increments its counter:

$$a : 6 \rightarrow 7$$

Therefore, message m_3 carries timestamp 7.

After that, process a receives message m_2 , which carries timestamp 6. Since the local counter of process a is already 7, the receive rule gives:

$$a := \max(7, 6) + 1 = 8$$

Next, process a sends message m_4 . Before sending, process a increments its counter again:

$$a : 8 \rightarrow 9$$

Therefore, message m_4 carries timestamp 9. The marked point on process a is after sending m_4 , so the value in the circle on process a is:

$$a = 9$$

To summarize the values are as the following:

- receive m_1 (timestamp 5): $\max(1, 5) + 1 = 6$.
- send m_3 : $6 \rightarrow 7$, hence m_3 carries timestamp 7.
- receive m_2 (timestamp 6): $\max(7, 6) + 1 = 8$.
- send m_4 : $8 \rightarrow 9$, hence m_4 carries timestamp 9.
- point in time: counter = 9 (box on a , after sending m_4).

Process c (start = 3):

Process c starts with counter value 3. First, process c receives message m_3 , which carries timestamp 7. Therefore, the local counter is updated as follows:

$$c := \max(3, 7) + 1 = 8$$

The marked point on process c is directly after receiving m_3 . Hence, the value in the circle on process c is:

$$c = 8$$

Afterwards, process c sends message m_5 . Before sending, it increments its local counter:

$$c : 8 \rightarrow 9$$

Therefore, message m_5 carries timestamp 9.

To summarize the values are as the following:

- receive m_3 (timestamp 7): $\max(3, 7) + 1 = 8$.
- point in time: counter = 8 (box on c , after receiving m_3).
- send m_5 : $8 \rightarrow 9$, hence m_5 carries timestamp 9.

Final result:

The message timestamps used above are summarised in the following Table 3, and the four requested values are $a = 9$, $b_{\text{mid}} = 6$, $b_{\text{right}} = 11$, and $c = 8$.

Message	m_1	m_2	m_3	m_4	m_5
Timestamp	5	6	7	9	9

Table 3: Lamport timestamps attached to each message in part (a).

However c has to receive m_3 before it can send m_5 , and that b has to receive m_4 before m_5 . Therefore the order of the receive events on b is fixed, which is why b first reaches 10 and then 11. The two messages m_4 and m_5 both carry the timestamp 9, but they originate from different processes (a and c respectively).

Solution 1b) – Vector clocks

For the vector clocks every process maintains a vector $V = (V_a, V_b, V_c)$ with one component per observed process. We write the vectors in the order (a, b, c) throughout.

In contrast to scalar clocks, a vector clock does not only store one timestamp, but also contains information about what a process knows about the progress of the other processes.

The update rules are:

- Before a process P_i executes a *send*, it increments its own component V_i by one and attaches the whole vector to the message.
- On *receiving* a message with vector W , the process first takes the component-wise maximum $V_k \leftarrow \max(V_k, W_k)$ for all k , and then increments its own component V_i by one.

The initial vectors provided by the diagram are:

- $A = (3, 1, 2)$
- $B = (2, 4, 3)$
- $C = (5, 2, 1)$

As in part (a), we evaluate the events in causal order so that every message is sent before it is received. We also keep the local order on each process line from left to right.

To do so, we use the messages directions from the diagram. The 5 messages are as follows:

Message	Sender	Receiver
m1	process b	process c
m2	process a	process c
m3	process c	process a
m4	process b	process a
m5	process c	process b

Table 4: Message directions

Process b (start $(2, 4, 3)$):

Process b starts with vector clock $(2, 4, 3)$. First, process b sends message m_1 . Before sending, it increments its own component, which is the second component in the vector:

$$(2, 4, 3) \rightarrow (2, 5, 3)$$

Therefore, message m_1 carries vector timestamp $(2, 5, 3)$. At this point, the middle marked point on process b is reached. Hence, the value in the middle box on process b is:

$$b_{mid} = (2, 5, 3)$$

Next, process b sends message m_4 . Again, process b increments its own component before sending:

$$(2, 5, 3) \rightarrow (2, 6, 3)$$

Therefore, message m_4 carries vector timestamp $(2, 6, 3)$.

Finally, process b receives message m_5 . Message m_5 carries vector timestamp $(5, 5, 7)$, as derived below from process c . The local vector of process b before receiving m_5 is $(2, 6, 3)$. First, we take the component-wise maximum:

$$\max((2, 6, 3), (5, 5, 7)) = (5, 6, 7)$$

After that, process b increments its own component by one:

$$(5, 6, 7) \rightarrow (5, 7, 7)$$

Thus, the value in the right box on process b is:

$$b_{right} = (5, 7, 7)$$

To summarize the values are as follows:

- send m_1 : $(2, 4, 3) \rightarrow (2, 5, 3)$; m_1 carries $(2, 5, 3)$.
- point in time: $(a: 2, b: 5, c: 3)$ (middle box on b).
- send m_4 : $(2, 5, 3) \rightarrow (2, 6, 3)$; m_4 carries $(2, 6, 3)$.
- receive m_5 $(5, 5, 7)$: $\max \rightarrow (5, 6, 7)$, then $+1$ on $b \rightarrow (5, 7, 7)$.
- point in time: $(a: 5, b: 7, c: 7)$ (right box on b).

Process a (start $(3, 1, 2)$):

Process a starts with vector clock $(3, 1, 2)$. First, process a sends message m_2 . Before sending, it increments its own component, which is the first component in the vector:

$$(3, 1, 2) \rightarrow (4, 1, 2)$$

Therefore, message m_2 carries vector timestamp $(4, 1, 2)$.

After that, process a receives message m_3 . Message m_3 carries vector timestamp $(5, 5, 6)$, as derived below from process c . The local vector of process a before receiving m_3 is $(4, 1, 2)$. First, we take the component-wise maximum:

$$\max((4, 1, 2), (5, 5, 6)) = (5, 5, 6)$$

Then process a increments its own component by one:

$$(5, 5, 6) \rightarrow (6, 5, 6)$$

Next, process a receives message m_4 , which carries vector timestamp $(2, 6, 3)$. The local vector of process a is currently $(6, 5, 6)$. Therefore, we again take the component-wise maximum:

$$\max((6, 5, 6), (2, 6, 3)) = (6, 6, 6)$$

Afterwards, process a increments its own component by one:

$$(6, 6, 6) \rightarrow (7, 6, 6)$$

The marked point on process a is after receiving m_4 . Hence, the value in the box on process a is:

$$a = (7, 6, 6)$$

To summarize, the values are as follows:

- send m_2 : $(3, 1, 2) \rightarrow (4, 1, 2)$; m_2 carries $(4, 1, 2)$.
- receive m_3 $(5, 5, 6)$: $\max \rightarrow (5, 5, 6)$, then $+1$ on $a \rightarrow (6, 5, 6)$.
- receive m_4 $(2, 6, 3)$: $\max \rightarrow (6, 6, 6)$, then $+1$ on $a \rightarrow (7, 6, 6)$.
- point in time: $(a: 7, b: 6, c: 6)$ (box on a).

Process c (start $(5, 2, 1)$):

Process c starts with vector clock $(5, 2, 1)$. First, process c receives message m_1 , which carries vector timestamp $(2, 5, 3)$. Therefore, process c first takes the component-wise maximum:

$$\max((5, 2, 1), (2, 5, 3)) = (5, 5, 3)$$

Then process c increments its own component, which is the third component:

$$(5, 5, 3) \rightarrow (5, 5, 4)$$

Next, process c receives message m_2 , which carries vector timestamp $(4, 1, 2)$. Since the local vector of process c is already $(5, 5, 4)$, the component-wise maximum does not change the vector:

$$\max((5, 5, 4), (4, 1, 2)) = (5, 5, 4)$$

After the receive event, process c increments its own component by one:

$$(5, 5, 4) \rightarrow (5, 5, 5)$$

Afterwards, process c sends message m_3 . Before sending, process c increments its own component:

$$(5, 5, 5) \rightarrow (5, 5, 6)$$

Therefore, message m_3 carries vector timestamp $(5, 5, 6)$. At this point, the marked point on process c is reached. Hence, the value in the box on process c is:

$$c = (5, 5, 6)$$

Finally, process c sends message m_5 . Before sending, process c increments its own component again:

$$(5, 5, 6) \rightarrow (5, 5, 7)$$

Therefore, message m_5 carries vector timestamp $(5, 5, 7)$.

To summarize, the values are as follows:

- receive m_1 (2, 5, 3): $\max \rightarrow (5, 5, 3)$, then $+1$ on $c \rightarrow (5, 5, 4)$.
- receive m_2 (4, 1, 2): $\max \rightarrow (5, 5, 4)$, then $+1$ on $c \rightarrow (5, 5, 5)$.
- send m_3 : $(5, 5, 5) \rightarrow (5, 5, 6)$; m_3 carries (5, 5, 6).
- point in time: ($a: 5$, $b: 5$, $c: 6$) (box on c).
- send m_5 : $(5, 5, 6) \rightarrow (5, 5, 7)$; m_5 carries (5, 5, 7).

Final result:

The vector timestamps used above are summarized in the following table:

Message	m_1	m_2	m_3	m_4	m_5
Timestamp	(2, 5, 3)	(4, 1, 2)	(5, 5, 6)	(2, 6, 3)	(5, 5, 7)

Table 5: Vector timestamps of the messages

The four requested vectors are therefore:

$$a = (7, 6, 6), \quad b_{mid} = (2, 5, 3), \quad b_{right} = (5, 7, 7), \quad c = (5, 5, 6)$$

However, when a receives m_3 , its own component jumps from 4 directly to 6 instead of increasing by a single step. The reason is that m_3 was sent by c , whose initial vector already contained $a = 5$, and therefore the component-wise maximum lifts a 's own entry to 5 before the receive increment adds one.

Exercise 2 (Denormalization) [2 Points]

Consider a hotel room booking system implemented in a traditional RDBMS and assume that you want to migrate this system to use a document store using JSON representation instead. The relational model has four tables: **room**, **booking**, **occupancy**, and **guest**. An example instance is given below. Conceive a data model for your document store system and translate the example instance to JSON files fitting your data model. You are allowed to add new attributes, denormalize relations, and make other similar changes for your new data model.

As discussed in the lecture, data models are designed with specific workloads in mind. Trade-offs may be necessary; if so, describe them in your report. For this exercise, consider the following parameters:

- The following query should be fast:
 - Listing the names of guests who stayed at the hotel in a given year.
- The **booking** relation is updated frequently and therefore needs to be efficient with respect to updates. For all other relations, updates are rare and can be more expensive.

room			booking			
id	type	price	guest	room	nights	check_in
1	Single	89	3	1	3	05-10-2025
2	Double	149	12	2	7	12-10-2024
3	Suite	299	15	3	2	20-03-2025
			15	3	5	30-09-2024
			18	1	10	01-03-2025

occupancy			guest	
room	year	available_nights	id	name
1	2024	280	3	Bob
1	2025	310	12	Alice
2	2025	240	15	Tom
3	2024	180	18	Ben
3	2025	195		

Table 6: Initial relational model instance

Solution 2)

Design idea. The central design decision in a document store is how far to denormalize, and this decision follows directly from the workload. We therefore use the following three collections:

- rooms
- guests
- bookings

We then shape each of them according to how it is read and written. The guiding principle is that data which is read together should be stored together, as long as this does not make the frequently updated documents expensive to write.

The bookings collection.

Because the **booking** relation is updated frequently, every booking is kept as its own small, standalone document. This is the most important choice in the whole model: a booking is never embedded inside a guest or a room document, because embedding it there would force a rewrite of a large parent document on every booking change.

Keeping bookings separate means that an insert or update only touches one small document.

In order to make the required query fast, each booking document additionally carries two denormalized fields.

First, the **guest's name** is stored directly as **guest_name**, so that the query for the guests of a given year can be answered from the **bookings** collection alone, without a lookup into **guests**.

Second, a derived field **year** is precomputed from the check-in date, so that the query can filter on an indexed integer field instead of parsing date strings at query time.

With an index on **year**, listing the guests of a given year then reduces to a single indexed scan over **bookings** followed by a projection on **guest_name**, as the following sample query illustrates:

The required query can then be answered directly from the **bookings** collection. A simple version would be:

```
db.bookings.distinct("guest_name", { year: 2025 })  
// -> ["Bob", "Tom", "Ben"]
```

```
{  
  "guest_id": 3, "guest_name": "Bob", "room_id": 1,  
  "nights": 3, "check_in": "2025-10-05", "year": 2025  
}
```

For a more precise version, especially in case two different guests have the same name, the query can group by `guest_id` and return the corresponding name:

```
db.bookings.aggregate([
  { $match: { year: 2025 } },
  {
    $group: {
      _id: "$guest_id",
      guest_name: { $first: "$guest_name" }
    }
  },
  {
    $project: {
      _id: 0,
      guest_name: 1
    }
  }
])
```

For the given example data, the result for the year 2025 is:

```
[
  { "guest_name": "Bob" },
  { "guest_name": "Tom" },
  { "guest_name": "Ben" }
]
```

The rooms collection. A room and its yearly occupancy figures are always read together and are updated only rarely, which is why the `occupancy` table is embedded as an array inside the corresponding room document. This removes one collection entirely and turns what would be a join in the relational model into a single document read. Since occupancy is rarely updated, the slightly higher cost of rewriting a room document on an occupancy change is acceptable.

```
{
  "_id": 1, "type": "Single", "price": 89,
  "occupancy": [
    { "year": 2024, "available_nights": 280 },
    { "year": 2025, "available_nights": 310 }
  ]
}
```

The guests collection. Guests are kept as their own documents and remain the single source of truth for guest names. They are referenced from bookings by `guest_id`, which is necessary so that a guest who has never booked, or for whom we need master data, still exists independently of any booking.

```
{ "_id": 3, "name": "Bob" }
```

Trade-offs. The model trades a controlled amount of redundancy for read performance, and the trade-offs are the following:

- The guest name is duplicated into every booking of that guest. This is the price for answering the year query without a join. Since guest names change very rarely (and updates to `guest` are explicitly stated to be rare), the cost of having to update several bookings when a name changes is acceptable, whereas the benefit, a join-free, indexable read for the frequent query, applies to the workload that matters.

- The **year** field is redundant with **check_in**. We accept this because it allows an index on a clean integer and avoids date parsing at query time. We additionally normalized the check-in dates to the ISO format YYYY-MM-DD, which makes them sort correctly as strings and keeps the derived **year** consistent.
- Embedding **occupancy** into **rooms** couples the two, which is only sensible because occupancy is rarely updated; for a write-heavy occupancy we would instead keep it as a separate collection.

Assumption. We define “a guest stayed in year Y ” as the check-in date falling into year Y . A stay that spans the turn of the year is therefore attributed to the year of its check-in. If multi-year stays had to be covered exactly, the derived field could be generalized from a single **year** to a list of covered years without changing the rest of the model.

Exercise 3 (Graph Databases (Neo4j and Cypher)) [4 Points]

For this exercise we work with a subset of the Wikidata knowledge graph that is loaded into Neo4j through the `neosemantics` plugin. The graph contains the node labels `Person`, `University`, `Award`, `Language`, `Place`, `Region`, and `Country`, connected by the relationship types `STUDIED_AT`, `EMPLOYED_AT`, `RECEIVED_AWARD`, `SPEAKS_LANGUAGE`, `BORN_IN`, and `LOCATED_IN`.

Solution 3.1)

(i) Languages of awarded persons. We match every person who speaks a language and keep only those for whom at least one `RECEIVED_AWARD` edge exists. The `EXISTS` subquery is used as the filter, and `DISTINCT` removes the languages that occur more than once.

```
MATCH (p:Person)-[:SPEAKS_LANGUAGE]->(l:Language)
WHERE EXISTS { (p)-[:RECEIVED_AWARD]->() }
RETURN DISTINCT l.name AS language
ORDER BY language;
```

The query returns **70** distinct languages. A few representative entries are *English*, *German*, *French*, *Russian*, *Japanese*, *Latin*, *Hebrew*, *Esperanto*, and *Yiddish*; the full list is sorted alphabetically and ranges from *American English* to *renaissance Latin*.

(ii) Countries with a studied-at university. Next, we follow the path from a person over the university to the country and return the distinct country names. Since the `LOCATED_IN` relation is also used between places and regions, we anchor the pattern explicitly at *University* and *Country*, which is why only universities that are actually located in a country are considered.

```
MATCH (:Person)-[:STUDIED_AT]->(:University)-[:LOCATED_IN]->(c:Country)
RETURN DISTINCT c.name AS country
ORDER BY country;
```

This returns **98** distinct countries, including current states such as *Austria*, *Germany*, *United States* and *Japan*, as well as historical entities that still exist as separate nodes in Wikidata, for example *Austria-Hungary*, *German Reich*, *Kingdom of Prussia* and *Soviet Union*.

(iii) Persons per country. We keep the same pattern but now count the distinct persons per country. Counting `DISTINCT p` is necessary because a person who studied at two universities in the same country must only be counted once.

```
MATCH (p:Person)-[:STUDIED_AT]->(:University)-[:LOCATED_IN]->(c:Country)
RETURN c.name AS country, count(DISTINCT p) AS persons
ORDER BY persons DESC, country;
```

Country	Persons	Country	Persons
United States	28 034	Israel	1 664
United Kingdom	20 939	P.R. China	1 112
Sweden	11 284	Canada	914
Austria	9 723	Ukraine	722
Italy	9 593	Japan	707
Netherlands	3 876	Romania	674
Spain	3 088	Germany	669
Switzerland	2 032	Belgium	499
Poland	1 775	Finland	489

Table 7: The 18 countries with the most students (out of 98 in total).

(iv) **Persons who additionally speak at least two languages.** Finally, we extend the previous pattern by a filter that keeps only persons with at least two distinct `SPEAKS_LANGUAGE` edges. The `COUNT` subquery counts these edges directly; since the data contains no duplicate language edges, this is the same as counting distinct languages.

```
MATCH (p:Person)-[:STUDIED_AT]->(:University)-[:LOCATED_IN]->(c:Country)
WHERE COUNT { (p)-[:SPEAKS_LANGUAGE]->(:Language) } >= 2
RETURN c.name AS country, count(DISTINCT p) AS persons
ORDER BY persons DESC, country;
```

Country	Persons	Country	Persons
United States	272	United Kingdom	59
Netherlands	210	Romania	56
Italy	184	Hungary	31
Poland	147	Greece	26
Czech Republic	112	Finland	24
Austria	107	Germany	20
Israel	94	P.R. China	20
Sweden	76	Soviet Union	20
Japan	62	Belgium	19

Table 8: Top countries by persons who studied there and speak ≥ 2 languages.

The ordering changes considerably compared to (iii): although the United States still leads, the Netherlands, Italy and Poland move far up, which can be attributed to the fact that multilingualism is recorded much more densely for European persons in Wikidata than the raw student counts would suggest.

Solution 3.2)

(i) **Top 10 universities by employees.** For this query we match all incoming EMPLOYED_AT edges of every university and count them. The exercise asks explicitly for the number of *edges*, which is why we use `count(e)` on the relationship and not `count(DISTINCT p)`.

```
MATCH (u:University)<-[e:EMPLOYED_AT]-(:Person)
RETURN u.name AS university, count(e) AS employed_edges
ORDER BY employed_edges DESC
LIMIT 10;
```

University	Edges
Massachusetts Institute of Technology	297
Leiden University	295
Princeton University	276
University of California, Los Angeles	261
University of California, Berkeley	240
Harvard University	233
University of Michigan	216
Institute for Advanced Study	208
Delft University of Technology	206
Stanford University	196

Table 9: The 10 universities with the most incoming EMPLOYED_AT edges.

(ii) **Top 10 regions by persons.** Next, we take the region exactly one level above the place of birth, that is the path `Person -> Place -> Region`, and count the distinct persons per region.

```
MATCH (p:Person)-[:BORN_IN]->(:Place)-[:LOCATED_IN]->(r:Region)
RETURN r.name AS region, count(DISTINCT p) AS persons
ORDER BY persons DESC
LIMIT 10;
```

Region	Persons	Region	Persons
New York	519	Grand Paris	180
New York City	275	Île-de-France	157
Saxony	252	arrondissement of Paris	157
Cook County	250	Kingdom of France	157
Greater London	183	Amsterdam	142

Table 10: The 10 regions one level above the place of birth with the most persons.

(iii) **Top 3 countries for Turing Award + ACM Fellow holders.** We look for persons who were employed at a university in some country and who received *both* the “Turing Award” and the “ACM Fellow” award. Both conditions are expressed as separate **EXISTS** subqueries so that the two awards are required simultaneously.

```
MATCH (p:Person)-[:EMPLOYED_AT]->(:University)-[:LOCATED_IN]->(c:Country)
WHERE EXISTS { (p)-[:RECEIVED_AWARD]->(:Award {name: 'Turing Award'}) }
      AND EXISTS { (p)-[:RECEIVED_AWARD]->(:Award {name: 'ACM Fellow'}) }
RETURN c.name AS country, count(DISTINCT p) AS persons
ORDER BY persons DESC
LIMIT 3;
```

Country	Persons
United States	3
Netherlands	1
United Kingdom	1

Table 11: Countries with the most persons holding both awards who were employed at a university there.

Even though 57 persons in the whole graph hold both awards, only few of them are also connected to a university with a known country through an **EMPLOYED_AT** edge, which is why the counts are small.

Solution 3.3) – Subgraph search

The subgraph from Figure 3 consists of two different persons who were both born in the same place, who both studied at the same university, and who each received at least one award. We translate this shape directly into a single **MATCH** pattern: the central part links the two persons through the shared university, the second line of the pattern links them through the shared place, and the two **RECEIVED_AWARD** edges attach an award to each person.

```
MATCH (a1:Award)<-[:RECEIVED_AWARD]-(p1:Person)-[:STUDIED_AT]->(u:University)
      <-[:STUDIED_AT]-(p2:Person)-[:RECEIVED_AWARD]->(a2:Award),
      (p1)-[:BORN_IN]->(p1:Place)<-[:BORN_IN]-(p2)
WHERE p1.uri < p2.uri
RETURN p1, p2, u, p1, a1, a2
LIMIT 1;
```

The condition `p1.uri < p2.uri` guarantees that the two matched persons are genuinely different and at the same time makes the result deterministic, because each unordered pair is reported only once. The first match returned by Neo4j is the subgraph shown in Figure 2: the physicists *William Feldman* and *Kenneth Freeman* were both born in *Perth*, both studied at the *University of Western Australia*, and each received an award.

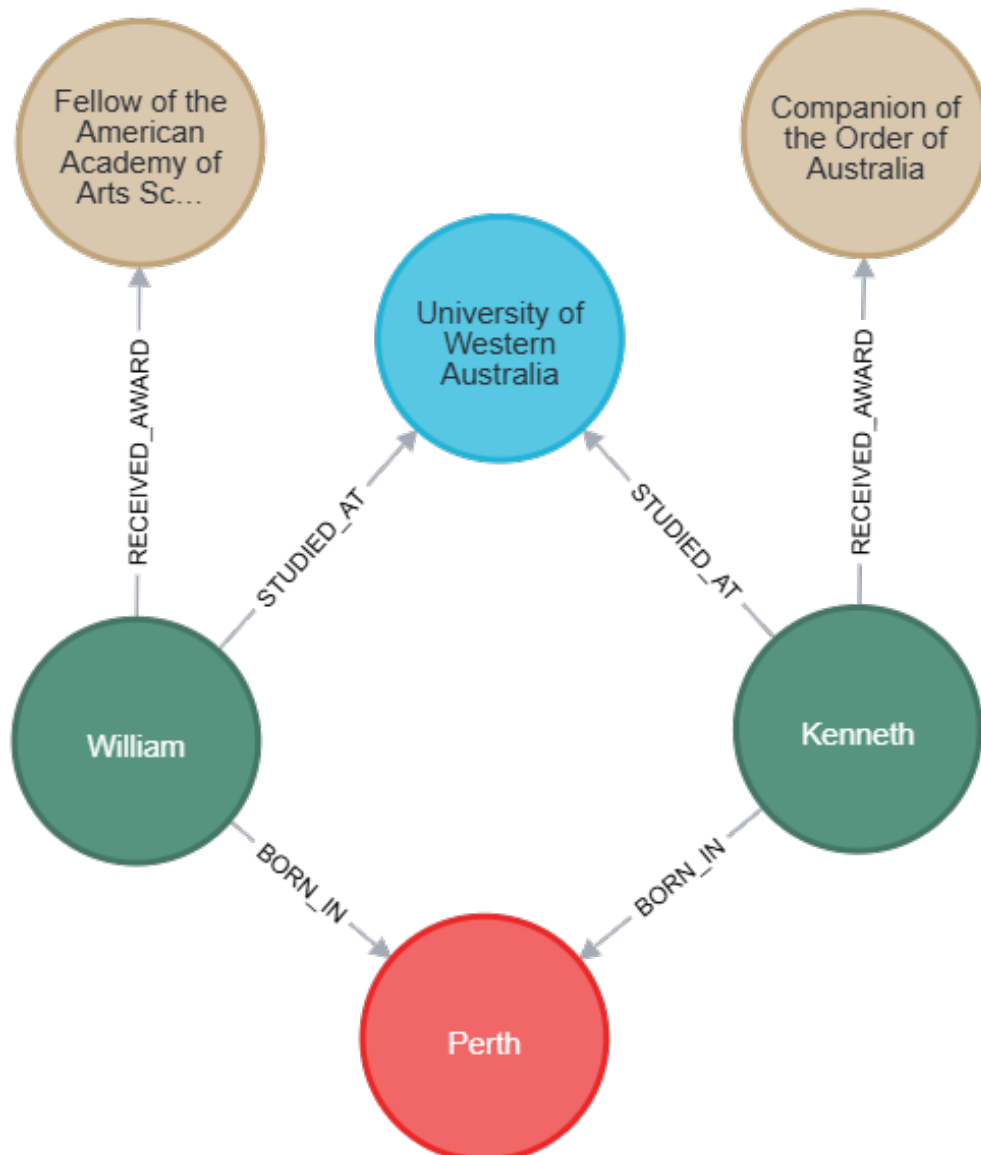


Figure 1: One subgraph of the shape of Figure 3, as returned by the querywith firstname.

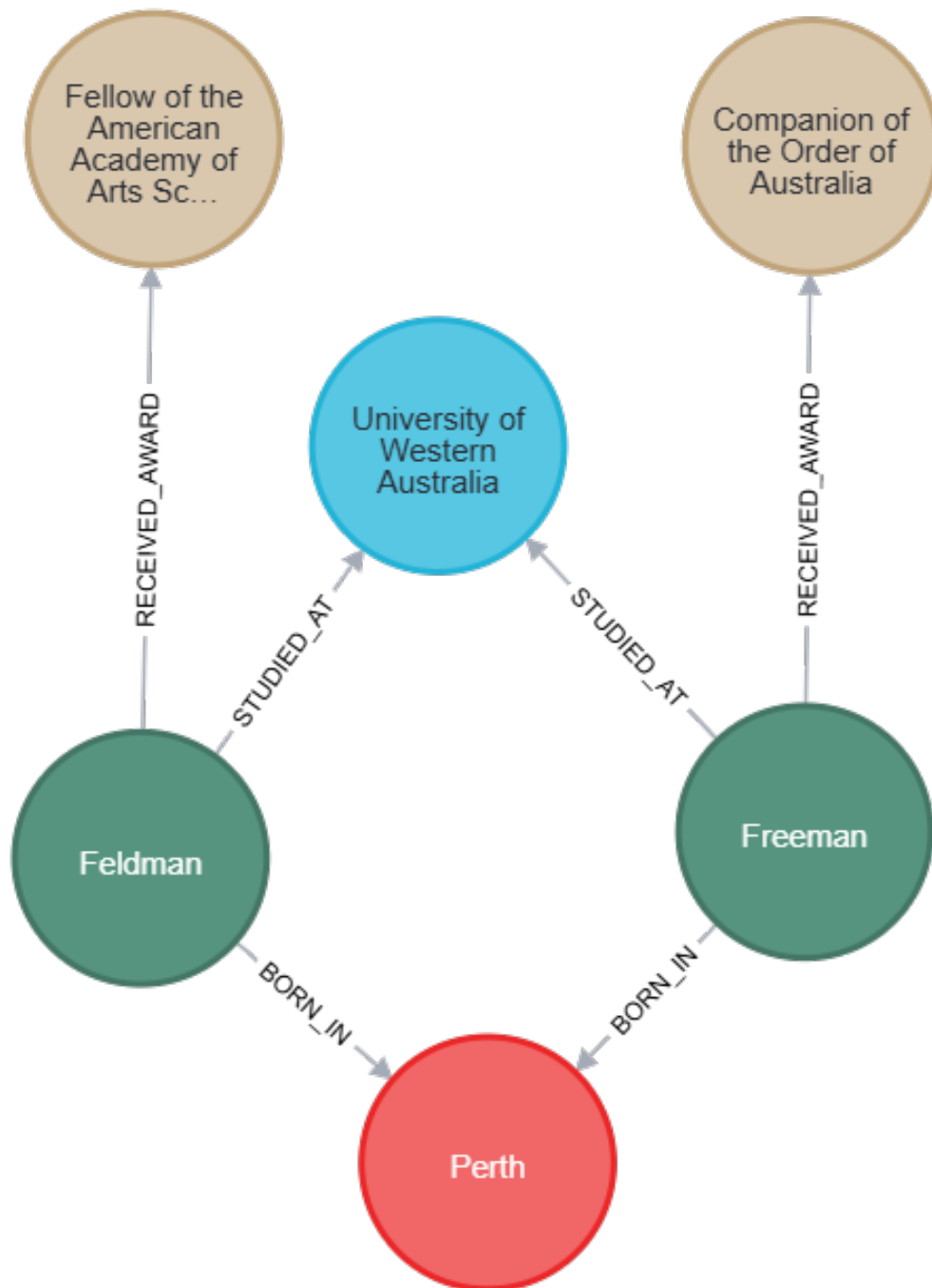


Figure 2: One subgraph of the shape of Figure 3, as returned by the query with lastname.

Solution 3.4) – Externalising names and dates with MERGE

The goal of this query is to turn two attributes of the persons into real nodes of the graph. In the given dataset the attribute that holds a person’s family name is `last_name` (only six persons carry a generic `name` property, whereas about 81 000 carry `last_name`), which is why we treat `last_name` as the “name” of the task. The same applies analogously to `date_of_birth`.

The key tool here is **MERGE**. According to the lecture, **MERGE** either matches an existing pattern or creates it if it does not yet exist, and it therefore behaves like a combination of **MATCH** and **CREATE**. This is what we need to avoid duplicate nodes and duplicate edges: merging on `(n:Name {name: p.last_name})` reuses the same **Name** node for every person with the same family name, and merging the edge afterwards never creates a second identical edge. We restrict the processing to persons with at least one award, and we use a **FOREACH** over a **CASE** list to perform the merge only when the corresponding attribute is present – if the attribute is missing, the list is empty and nothing happens for that person.

```
MATCH (p:Person)
WHERE EXISTS { (p)-[:RECEIVED_AWARD]->() }
FOREACH (_ IN CASE WHEN p.last_name IS NOT NULL THEN [1] ELSE [] END |
  MERGE (n:Name {name: p.last_name})
  MERGE (p)-[:HAS_NAME]->(n))
FOREACH (_ IN CASE WHEN p.date_of_birth IS NOT NULL THEN [1] ELSE [] END |
  MERGE (d:DateOfBirth {date_of_birth: p.date_of_birth})
  MERGE (p)-[:HAS_DOB]->(d));
```

Running this statement creates 7 772 **HAS_NAME** edges that point to only 6 247 distinct **Name** nodes, and 7 604 **HAS_DOB** edges pointing to 6 318 distinct **DateOfBirth** nodes. The fact that there are clearly fewer nodes than edges confirms that **MERGE** correctly shared the nodes for family names and birth dates that occur more than once.

The five most common family names. To show the persons that are connected to the five most frequent family names, we first determine these five **Name** nodes by counting their incoming **HAS_NAME** edges, and then we match all persons attached to them. Returning the persons together with the edge and the **Name** node lets the Neo4j browser draw the corresponding star-shaped subgraphs.

```
MATCH (:Person)-[:HAS_NAME]->(n:Name)
WITH n, count(*) AS persons
ORDER BY persons DESC
LIMIT 5
MATCH (p:Person)-[:HAS_NAME]->(n)
RETURN p, r, n;
```

Family name	Persons
Brown	18
Smith	18
Johnson	17
Cohen	16
Wilson	14

Table 12: The five most common family names among the processed (awarded) persons.

Solution 3.5) – Shortest paths on the extended graph

The extended graph introduces a new connection between two universities u and v whenever there is a person p_1 who studied at both of them and a *different* person p_2 who was employed at both of them. As suggested in the exercise, we materialise this relationship as a new edge type `CONNECTED` in a first step, because this turns the otherwise complex condition into ordinary edges over which the `shortestPath` function can then operate directly.

```
MATCH (p1:Person)-[:STUDIED_AT]->(u:University), (p1)-[:STUDIED_AT]->(v:University)
WHERE u <> v
MATCH (p2:Person)-[:EMPLOYED_AT]->(u), (p2)-[:EMPLOYED_AT]->(v)
WHERE p2 <> p1
MERGE (u)-[:CONNECTED]-(v);
```

Here, we use `MERGE` again so that each university pair is connected by exactly one edge, and we merge the edge *undirected* because the connection is symmetric. The condition $u \neq v$ excludes self-loops and $p2 \neq p1$ enforces that the two persons are different, as required. This step creates 2681 `CONNECTED` edges between universities.

In a second step we compute, for the start university “TU Wien”, a shortest path to every university located in the country “Germany”. The `shortestPath` function returns one shortest path *per matched destination* rather than a single global shortest path, which is exactly what we want here.

```
MATCH (start:University {name: 'TU Wien'})
MATCH (dest:University)-[:LOCATED_IN]->(:Country {name: 'Germany'})
MATCH path = shortestPath((start)-[:CONNECTED*]-(dest))
RETURN [n IN nodes(path) | n.name] AS path_universities, length(path) AS hops
ORDER BY hops, dest.name;
```

Although 18 universities are located in Germany, only two of them are reachable from “TU Wien” through `CONNECTED` edges, because this newly constructed graph is sparse and decomposes into several components. The two reachable universities are both found at a distance of three hops:

Destination (Germany)	Shortest path (3 hops)
University of Wittenberg	TU Wien – Wrocław – Leipzig – Wittenberg
University of Helmstedt	TU Wien – Graz – Halle-Wittenberg – Helmstedt

Table 13: Shortest paths from TU Wien to the reachable German universities.

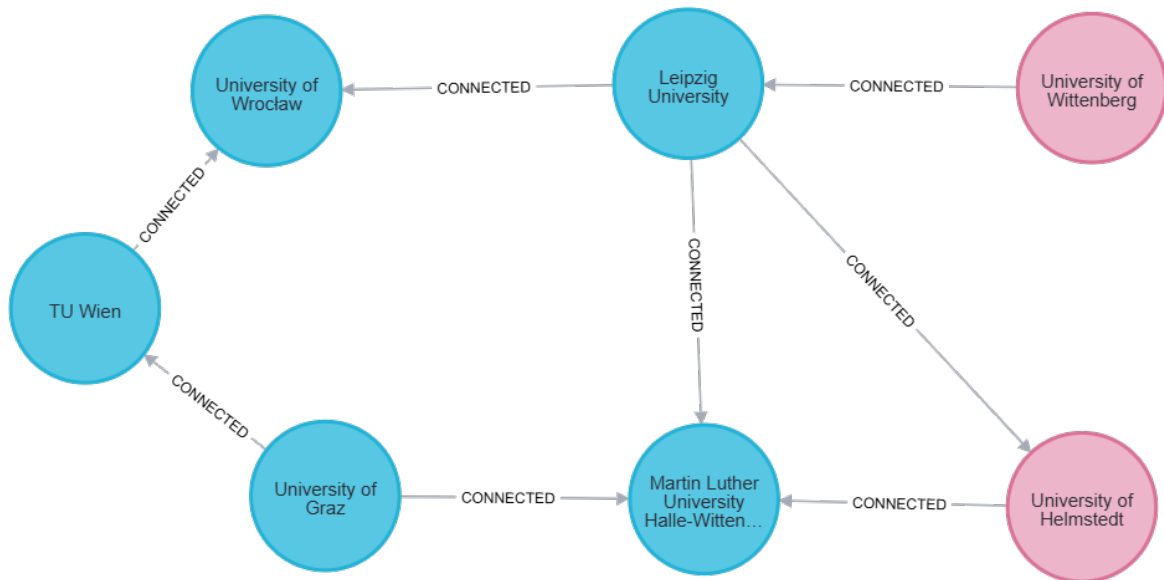


Figure 3: The two shortest paths found by `shortestPath`; the German destination universities are highlighted in red.

Finally, it is important to mention that `shortestPath` silently omits every destination that is not reachable at all, which is why the result contains only two rows instead of eighteen. The remaining German universities simply lie in different connected components of the `CONNECTED` graph and therefore have no path from TU Wien.

Exercise 4 (Column Stores: Late Materialization Query Plan) [2 points]

Consider the following query executed on a **column store** that uses **late materialization**:

```
SELECT sum(R.a) FROM R, S WHERE R.b = S.a AND 5 <= R.a AND R.a <= 20
```

There are two relations R and S . Each column is represented as a list of *(position, value)* pairs:

Relation R			
Ra		Rb	
<i>pos</i>	<i>val</i>	<i>pos</i>	<i>val</i>
1	3	1	12
2	16	2	34
3	25	3	17
4	9	4	56
5	5	5	22
6	31	6	34

Relation S			
Sa		Sb	
<i>pos</i>	<i>val</i>	<i>pos</i>	<i>val</i>
1	34	1	7
2	22	2	4
3	17	3	9

Query Plan

The query is evaluated using the following query plan. All operations work on single columns.

```

1. inter1 = select(Ra, 5, 20)
2. inter2 = reconstruct(Rb, inter1)
3. join_input_S = reverse(Sa)
4. join_res = join(inter2, join_input_S)
5. inter3 = voidTail(join_res)
6. inter4 = reconstruct(Ra, inter3)
7. result = sum(inter4)
```

Operation reference:

- `select(col, low, high)` – returns a position list (list of IDs) of all entries in `col` where $low \leq val \leq high$.
- `reconstruct(col, poslist)` – fetches values from `col` at each position in `poslist`; returns *(pos, val)* pairs.
- `reverse(col)` – swaps each *(pos, val)* to *(val, pos)*; used to build a lookup structure where value is key.
- `join(left, right)` – joins `left` values against `right` keys; returns *(pos_L, pos_R)* pairs.
- `voidTail(pairs)` – keeps only the left element of each pair.

Solution 4)

Step 1: `inter1 = select(Ra, 5, 20)`. The `select` operator returns the *positions* of all *R.a* entries whose value lies in $[5, 20]$. The qualifying values are 16 (position 2), 9 (position 4) and 5 (position 5); the values 3, 25 and 31 are outside the range.

Step 2: `inter2 = reconstruct(Rb, inter1)`. We fetch the *R.b* values at the positions from `inter1`, which yields $(pos, R.b)$ pairs. This is needed because the join in the query is on *R.b*.

Step 3: `join_input_S = reverse(Sa)`. The `reverse` operator swaps every (pos, val) of *S.a* into (val, pos) , so that the *value* becomes the key of a lookup structure. This prepares *S.a* for being used as the right (build) side of the join.

Step 1: `inter1`

pos
2
4
5

Step 2: `inter2`

pos	Rb
2	34
4	56
5	22

Step 3: `reverse(Sa)`

val	pos
34	1
22	2
17	3

Step 4: `join_res = join(inter2, join_input_S)`. The join matches the *R.b* values from `inter2` against the keys of the reversed *S.a* and returns (pos_L, pos_R) pairs. The value 34 at position 2 matches *S.a* position 1, and the value 22 at position 5 matches *S.a* position 2. The value 56 at position 4 has no partner and is dropped.

Step 5: `inter3 = voidTail(join_res)`. We keep only the left element of each pair, i.e. the positions in *R* that survived both the range predicate and the join.

Step 6: `inter4 = reconstruct(Ra, inter3)`. Only now, at the latest possible moment, do we fetch the actual *R.a* values that we want to aggregate. This is the defining property of late materialization: the values of the aggregation column are reconstructed only for the few positions that are guaranteed to be in the result.

Step 4: `join_res`

pos_L	pos_R
2	1
5	2

Step 5: `inter3`

pos
2
5

Step 6: `inter4`

pos	Ra
2	16
5	5

Step 7: `result = sum(inter4)`. Summing the two reconstructed values gives the final result.

Step 7: `result`

result
$16 + 5 = \mathbf{21}$

The final result is therefore **21**. We can verify this directly: the only *R* tuples that satisfy $5 \leq R.a \leq 20$ and whose *R.b* value occurs in *S.a* are the tuples at positions 2 (*R.a* = 16, *R.b* = 34 $\in$ *S.a*) and 5 (*R.a* = 5, *R.b* = 22 $\in$ *S.a*), and $16 + 5 = 21$.